

SahiRate — M12 Backend Handoff Specification

Project: SahiRate (SIH26229) **Problem Statement:** Kabadiwala Connect – Bringing the Informal Collector into the Formal Recycling Chain **Document Purpose:** Engineering specification and API contract for the M12 Backend implementation.

1. Introduction

What SahiRate is: SahiRate is a field-ready, offline-first platform designed to connect informal scrap collectors with the formal authorized recycling chain. The collector is the center of the system. SahiRate is **not** just a marketplace—it is an operating layer that handles offline lot creation, price discovery, localized audio guidance, verifiable QR-based handovers, and traceability.

Key Technical Differentiator: *Real offline-first operation.* The application works in poor-connectivity environments. Collectors can create lots, take photos, and process data completely offline. Synchronization happens deterministically when connectivity is restored.

2. Current Status (Frontend & Mock Data)

The frontend team has completed M1 through M11.4. The application is highly functional but currently runs on an **IndexedDB/Dexie local-first architecture** with mock/demo data for Admin features.

*[!WARNING] **LOCAL/DEMO ≠ PRODUCTION BACKEND.** The current frontend simulates authentication, synchronization, and admin data. The M12 backend must replace these simulations with real database-backed logic and enforce server-side constraints. Do **NOT** assume a feature is fully done just because the frontend renders it.*

Done / Mock / Remaining Matrix

AREA	FRONTEND STATUS	BACKEND STATUS	WHAT REMAINS
PWA & Offline Storage	DONE (Vite PWA, Dexie)	Not Started	Server-side sync endpoint
Collector UI	DONE (Offline-first)	Not Started	API endpoints to accept synced lots
Recycler UI	DONE (Dashboards, Accept)	Not Started	Authorization, State Machine, Matching
Admin UI	MOCK (Local Demo Adapter)	Not Started	Real Admin APIs replacing adminDemo.ts
Localization / Audio	DONE (EN, HI, MR)	N/A	None
Camera / Photos	DONE (Local Base64 storage)	Not Started	Image upload, validation, storage (S3/Local)
QR & Handover	MOCK (Local validation)	Not Started	Server-verifiable QR generation & validation
Payments	MOCK (Status tracking)	Not Started	Record keeping & ledger on backend

Authentication & Users	MOCK (UI Only)	Not Started	JWT Auth, Password hashing, Role enforcement
FastAPI & PostgreSQL	N/A	REMAINING	Full implementation
Offline Sync	DONE (Local Outbox)	Not Started	Ingest endpoint (POST /sync/events), Idempotency
AI / ML Classification	SKIPPED (Intentionally)	Not Started	Endpoint for Material Classification
Price Discovery	MOCK (Demo Data)	Not Started	Live Price API (distinguish live vs stale)
Recycler Matching	MOCK	Not Started	Algorithmic matching with explanation payload
Security	N/A	REMAINING	Password hashing, JWT, CORS, Idempotency
Cross-device functional	DONE (PWA/Responsive)	N/A	N/A

3. Frontend Architecture Overview

The frontend stack is **LOCKED**: React, Vite, TypeScript, Tailwind CSS, shadcn/ui, Dexie.js, Zustand.

Data Flow: UI -> Zustand Stores -> Dexie (IndexedDB) -> SyncManager (Outbox) -> [YOUR FASTAPI BACKEND] -> PostgreSQL

*[!IMPORTANT] **DO NOT** rewrite the frontend to be online-only. **DO NOT** remove Dexie/IndexedDB. The backend must respect this architecture and consume the outbox events.*

4. Core Workflows & State Transitions

Collector Flow

1. Collector opens SahiRate (even offline).
2. Creates collection lot (material, weight, price, photo).
3. Confirms lot -> Lot stored locally -> Outbox event LOT_CREATED.
4. Online sync pushes event to backend.
5. Recycler sees eligible lot and accepts it.
6. Handover QR generated by Recycler.
7. Collector scans QR, verifies amount, and confirms handover.
8. Recycler completes handover & payment is recorded.

Recycler Flow

1. Dashboard -> Available lots.
2. Accepts lot -> Accepted Lots / My Lots.
3. Verify lot (enter verified weight/final rate).
4. Generate handover QR.

5. Wait for collector confirmation.
6. Complete handover.
7. Record payment.

Admin Flow

Currently mocked via /admin (Transactions, Verification, Alerts, Audit). Backend must supply endpoints to replace deterministic demo states.

Lot State Machine (Server-Side Enforcement)

The backend must enforce valid state transitions. Never trust the frontend completely.

AVAILABLE -> ACCEPTED -> VERIFIED -> QR_GENERATED -> COLLECTOR_CONFIRMED -> COMPLETED

Invalid Transitions to Prevent:

- Accepting an already ACCEPTED lot.
- Completing a handover before COLLECTOR_CONFIRMED.
- Recording payment before completion.
- Changing ownership without authorization.

5. Database Architecture (Proposed PostgreSQL Schema)

Prefer PostgreSQL (+ PostGIS for location logic).

Key Tables:

- `users` (id UUID, phone, password_hash, role, created_at)
- `collector_profiles` / `recycler_profiles` (user_id UUID FK, location_geom, verified_status)
- `lots` (id UUID, collector_id UUID FK, material, approx_weight_kg, estimated_value, status, created_at_local)
- `lot_photos` (id UUID, lot_id UUID FK, object_url)
- `lot_acceptances` (id UUID, lot_id UUID FK, recycler_id UUID FK)
- `handovers` (id UUID, lot_id UUID FK, recycler_id UUID FK, final_weight, final_amount, qr_reference)
- `payments` (id UUID, handover_id UUID FK, amount, mode, status, timestamp)
- `audit_events` (id UUID, entity_id UUID, event_type, payload JSONB, actor_id UUID)

[!TIP] Ensure `idempotency_key` or `event_id` is stored to prevent duplicate outbox processing.

6. Authentication & Roles

- **Roles:** COLLECTOR, RECYCLER, ADMIN.
- **Strategy:** JWT-based authentication (Refresh/Access token pair recommended).
- **Claims Expected:** user_id, role, verified_status.

[!CAUTION] Do not trust frontend role selectors for authorization. The backend must explicitly enforce authorization checks on all protected routes.

7. Offline Sync API Contract

This is the most critical integration point. The frontend outbox will send queued events to the backend.

POST `/api/v1/sync/events`

- **Purpose:** Ingest offline events (e.g., LOT_CREATED).
- **Request Body:** Array of events.

```
{
  "events": [
    {
      "event_id": "uuid-1234",
      "type": "LOT_CREATED",
      "entity_id": "lot-uuid",
      "payload": { "material": "plastic", "weight": 10 },
      "created_at_local": "2026-09-12T10:00:00Z"
    }
  ]
}
```

- **Handling Requirements:**
 - Backend must check `event_id` against processed events (Idempotency).
 - Backend should apply the event to the database.
 - Return success/failure per event so the frontend SyncManager can clear successful events from the outbox.

GET /api/v1/sync/changes

- **Purpose:** Pull latest state from backend (for Recycler dashboards, price updates).

8. Proposed API Contract

Reconcile this with the frontend implementation during M12.

Auth & Users

- POST /api/v1/auth/register (ANY)
- POST /api/v1/auth/login (ANY)
- GET /api/v1/me (AUTH)

Lots & Recycler Actions

- POST /api/v1/lots (COLLECTOR)
- GET /api/v1/lots (REC/ADM)
- POST /api/v1/lots/{id}/accept (REC)
- POST /api/v1/lots/{id}/verify (REC)

Handovers & QR

- POST /api/v1/handovers (REC)
- POST /api/v1/handovers/{id}/confirm (COL) - Collector scans QR
- POST /api/v1/handovers/{id}/complete (REC)

Payments

- POST /api/v1/payments (REC)

Admin (Replaces `adminDemo.ts`)

- GET /api/v1/admin/transactions (ADM)

- GET /api/v1/admin/verification (ADM)
- POST /api/v1/admin/verification/{id}/approve (ADM)
- POST /api/v1/admin/verification/{id}/reject (ADM)
- GET /api/v1/admin/alerts (ADM)
- GET /api/v1/admin/audit (ADM)

API Error Contract Example

```
{
  "success": false,
  "error": {
    "code": "LOT_ALREADY_ACCEPTED",
    "message": "This lot has already been accepted.",
    "details": {}
  }
}
```

9. AI/ML, Price Discovery & Photos

- **AI/ML Integration:** Expose /api/v1/ai/classify . Input: Image. Output: {"material": "PET", "confidence": 0.92} . Do not fake confidence. Allow manual correction fallbacks.
- **Price Discovery:** The backend must serve live price maps (regional/material). The frontend needs to distinguish between cached prices (offline) and live prices. Do not present fake prices as production data.
- **Photos:** Implement an upload endpoint /api/v1/photos/upload . Ensure MIME validation and size limits.
- **Recycler Matching:** Provide explainable matching logic (distance, capacity, rate) rather than a black-box.

10. Security Checklist

- ☐ Password Hashing (bcrypt/argon2).
- ☐ JWT Validation on all protected endpoints.
- ☐ SQL Injection protection (use SQLAlchemy ORM).
- ☐ Idempotency keys on state-mutating requests (sync outbox).
- ☐ Ownership checks (Collector can only confirm THEIR handover).
- ☐ CORS properly configured for frontend URLs.
- ☐ Secure secrets via .env . No secrets committed.

11. Frontend -> Backend Integration Map

When building endpoints, track where they plug into the frontend:

- frontend/src/services/admin/adminDemo.ts -> Replace with /api/v1/admin/...
- frontend/src/services/syncManager.ts -> Plugs into /api/v1/sync/events
- frontend/src/pages/recycler/* -> Consumes lots , handovers , payments APIs.
- frontend/src/pages/collector/* -> Consumes lots , ai/classify , sync APIs.

12. Testing Requirements

Backend teammate must write:

- Unit & API tests.
- Idempotency tests (send same sync event twice, ensure 1 DB insert).
- State machine invalid-transition tests.
- **End-to-End SIH Demo Test:** Offline lot creation -> sync -> Recycler accept -> QR gen -> Collector confirm -> Payment.

13. FastAPI Project Structure (Proposed)

```
backend/
├── app/
│   ├── main.py
│   ├── core/ (config.py, security.py, database.py)
│   ├── models/ (SQLAlchemy models)
│   ├── schemas/ (Pydantic schemas)
│   ├── api/
│   │   └── v1/ (auth.py, users.py, lots.py, handovers.py, sync.py, admin.py, ai.py)
│   ├── services/
│   └── ml/
├── tests/
├── alembic/
└── requirements.txt
```

14. M12 Implementation Roadmap & "Start Here"

Implementation Order:

1. **M12.1 Foundation:** FastAPI, PostgreSQL, SQLAlchemy, Alembic, `/health` .
2. **M12.2 Auth:** Users, Roles, JWT, `/me` .
3. **M12.3 Core Domain:** Materials, Lots, Collectors, Recyclers.
4. **M12.4 Recycler Flow:** Available lots, Accept, State Machine.
5. **M12.5 Handover & QR:** Sessions, References, Confirmations.
6. **M12.6 Payments & Earnings:** Ledger entries.
7. **M12.7 Sync Engine:** Event ingestion (POST `/sync/events`), Idempotency.
8. **M12.8 Photos/Files:** Storage APIs.
9. **M12.9 Admin APIs.**
10. **M12.10 AI/ML:** Classification endpoints.
11. **M12.11 Frontend Integration.**

FIRST-DAY CHECKLIST FOR BACKEND TEAMMATE:

1. ☐ Pull the `main` branch.
2. ☐ Create and switch to `backend-dev` branch.
3. ☐ Inspect `frontend/src/services/syncManager.ts` and `frontend/src/services/admin/adminDemo.ts` .
4. ☐ Identify all local domain models in the TypeScript types.
5. ☐ Set up FastAPI and PostgreSQL environments.
6. ☐ Create SQLAlchemy models (Users, Lots, Handovers).

7. ☐ Configure Alembic.
8. ☐ Implement `/health` and basic Authentication.

*[!CAUTION] **DO NOT** rewrite the frontend. **DO NOT** remove Dexie/IndexedDB. **DO NOT** make the system online-only. **DO NOT** use fake production data. **DO NOT** implement a fake payment gateway.*